

MOVIE BOOKING SYSTEM

Project Report

VITyarthi - Build Your Own Project

A first-year university level Python + SQLite application

Student Name: Sujal Sharma

Registration Number: 26MIM10197

Course / Section: Integrated M.Tech Artificial Intelligence

Faculty: Abha Trivedi

Academic Year: 2026-27

Submitted as an academic project demonstrating programming, database, modular design, validation and testing concepts.

Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Functional Requirements
5. Non-functional Requirements
6. System Architecture
7. Design Diagrams
8. Design Decisions and Rationale
9. Implementation Details
10. Screenshots / Results
11. Testing Approach
12. Challenges Faced
13. Learnings and Key Takeaways
14. Future Enhancements
15. References

1. Introduction

The Movie Booking System is a small console-based application created to demonstrate how a real-world ticket booking problem can be solved using basic programming and database concepts. A customer can register, log in, browse movies, select a show, check available seats and make a booking. An administrator can manage movie and show information and view a simple booking report.

The project intentionally uses Python, SQLite and the standard library. This keeps the code understandable for a first-year university student while still showing modular design, CRUD operations, transactions, authentication, validation, logging and automated tests.

2. Problem Statement

Manual or unorganized movie ticket booking can make it difficult to track movies, show timings, seat availability and customer bookings. The project solves this by keeping the main information in a single SQLite database and providing a guided command-line workflow for customers and administrators.

3. Objectives

- Create a working movie ticket booking application with a clear user workflow.
- Apply modular programming using multiple Python files/modules.
- Store users, movies, shows, seats and bookings in a relational database.
- Prevent invalid input and avoid double-booking of seats.
- Provide basic administration and reporting features.
- Demonstrate testing using Python unittest and maintain the project in a Git-ready structure.

4. Functional Requirements

Module	Requirement	Input	Output
User Management	Register and log in	Name, email, password	Account / logged-in user
Movie Management	View, search, add, update and delete movies	Movie details / search text	Movie list or update result
Show Management	Add shows with screen, date, time and price	Movie ID, screen, date, time, price	Created show
Booking	View seats and book multiple seats	Show ID, seat numbers	Booking ID, seats, total amount
Booking Control	View history and cancel bookings	User ID, booking ID	Updated booking status
Reporting	View booking and revenue summary	Stored bookings	Counts, revenue and movie activity

5. Non-functional Requirements

Requirement	How it is addressed
Performance	SQLite queries are indexed for common lookups; the application is small and local.
Security	Passwords are stored as PBKDF2-HMAC-SHA256 derived hashes instead of plain text.
Usability	The CLI uses numbered menus and simple error messages suitable for a beginner user.
Reliability	Booking writes are wrapped in a database transaction and invalid

	bookings are rejected.
Maintainability	Responsibilities are separated into modules such as auth, booking, movies, reports and validation.
Error Handling	Input rules use validation functions and service-layer exceptions; database errors are converted into readable messages where needed.
Logging	Important application events such as startup and booking/cancellation are written to a log file.
Resource Efficiency	The project uses SQLite and only Python standard-library packages, so there is little setup overhead.

6. System Architecture

The system follows a simple layered structure rather than a large framework. The command-line interface receives input, service modules perform the main operations, validators check data, and the database module manages SQLite connections and the schema. This separation makes the project easier to read and test.

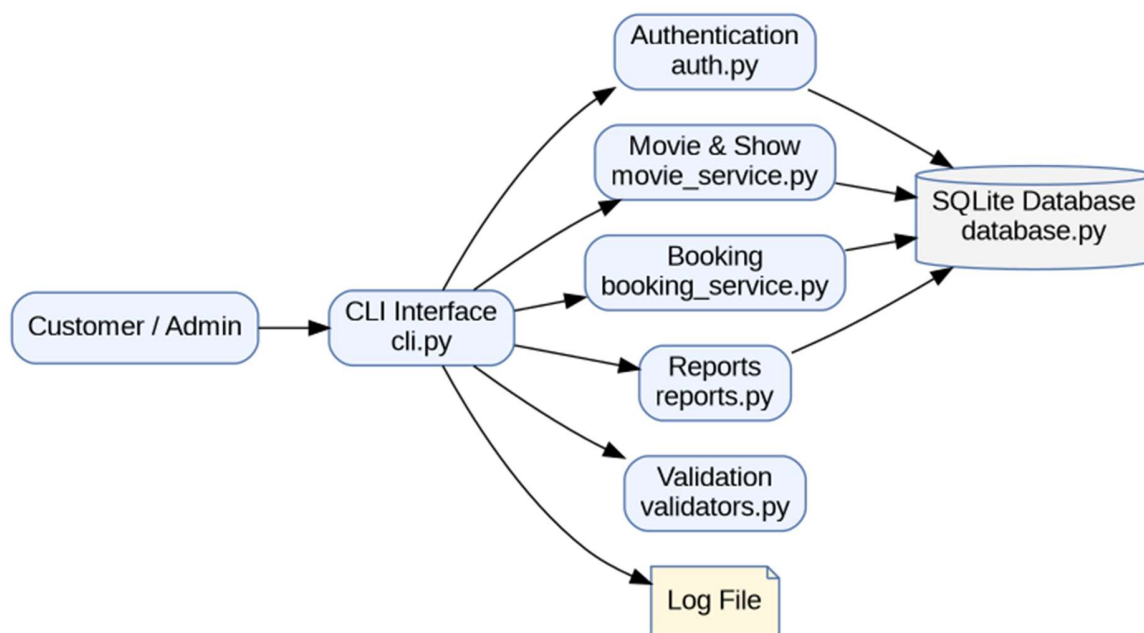


Figure 1. System Architecture

7. Design Diagrams

7.1 Workflow Diagram

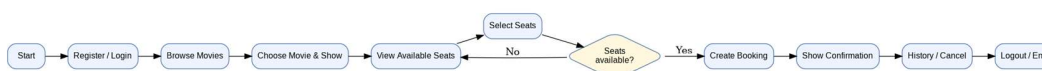


Figure 2. Customer Booking Workflow

The main flow starts with registration or login and continues through movie selection, show selection, seat checking and booking confirmation. If a selected seat is unavailable, the user is sent back to seat selection instead of creating an incorrect booking.

7.2 Use Case Diagram

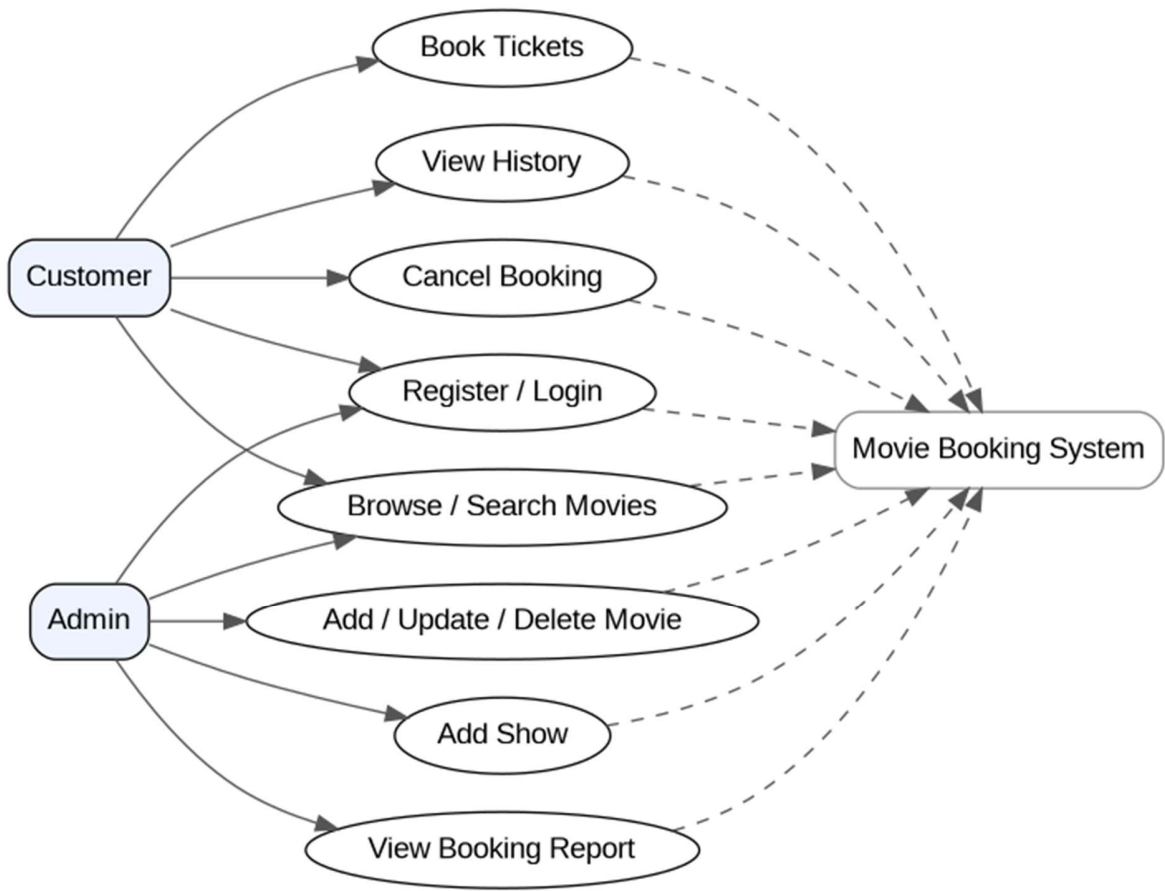


Figure 3. Use Case Diagram

7.3 Sequence Diagram

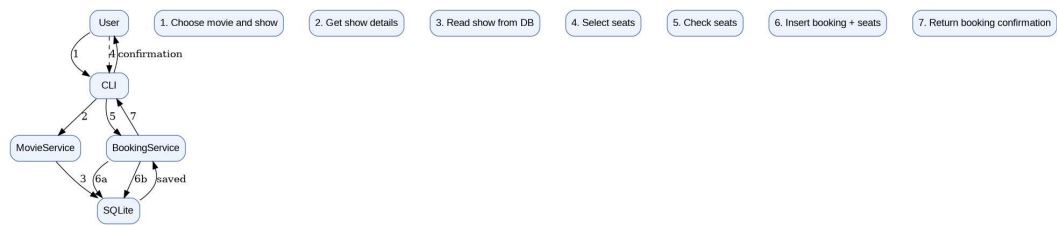


Figure 4. Booking Sequence

The sequence shows the interaction between the customer, CLI, movie service, booking service and SQLite database during a booking operation.

7.4 Class / Component Diagram

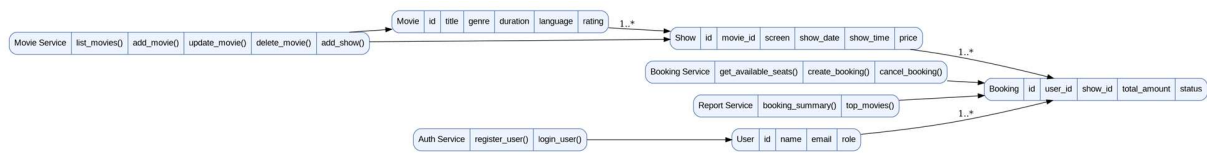


Figure 5. Class / Component Diagram

7.5 ER Diagram

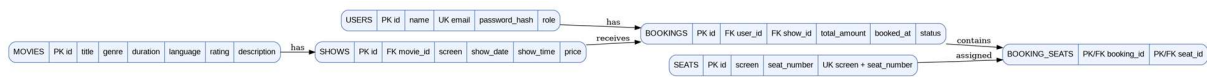


Figure 6. Entity Relationship Diagram

The many-to-many relationship between bookings and seats is represented using the BOOKING_SEATS bridge table. This lets one booking contain multiple seats while keeping individual seat records reusable across shows.

8. Design Decisions and Rationale

Decision	Reason
Python CLI	Simple enough for a first-year project and keeps the focus on programming logic.
SQLite	Provides real relational storage without requiring a separate database server.
Service modules	Separates authentication, movie/show logic, booking logic and reports.
Dataclasses	Provide simple structured objects for User, Movie, Show and Booking.
Transactions for bookings	Prevents a partial booking from being saved if something fails.
Seed data	Makes the first run easy to demonstrate without manually entering every movie and show.
unittest	Uses a built-in framework so no extra dependency is required.

9. Implementation Details

The project contains more than the required 5-10 meaningful files while keeping each file focused on one responsibility. The main modules are shown below.

File	Purpose
app.py	Simple entry point.
app/main.py	Initializes the database, logging and application.
app/database.py	Creates SQLite schema and connections.
app/models.py	Defines User, Movie, Show and Booking dataclasses.
app/auth.py	Registration, password hashing and login.
app/movie_service.py	Movie and show CRUD/management operations.
app/booking_service.py	Seat availability, booking, history and cancellation.
app/reports.py	Booking statistics, movie activity and revenue report.
app/validators.py	Reusable input validation functions.
app/cli.py	User/admin menus and interaction logic.
app/seed_data.py	Creates demo accounts, movies, shows and seats.
tests/test_system.py	Automated validation tests.
docs/diagrams.md	Mermaid versions of design diagrams.
README.md	Installation, run, testing and Git instructions.
statement.md	Problem statement, scope, users and high-level features.

Database Schema

Table	Important Fields	Purpose
users	id, name, email, password_hash, role	Stores accounts and roles.
movies	id, title, genre, duration, language, rating	Stores movie information.
shows	id, movie_id, screen, show_date, show_time, price	Stores screening details.
seats	id, screen, seat_number	Stores the physical seat layout for each screen.

bookings	id, user_id, show_id, total_amount, booked_at, status	Stores ticket orders and status.
booking_seats	booking_id, seat_id	Connects a booking with one or more seats.

Input Validation and Error Handling

- Email format is checked before registration or login.
- Passwords must contain at least six characters.
- Duration, rating, price, date and time inputs are validated before database insertion.
- The booking service checks for empty selections, duplicate selections, invalid seats and seats already booked.
- Users cannot cancel another user's booking.
- SQLite foreign keys and constraints provide an additional layer of data protection.

10. Screenshots / Results

The following sample output demonstrates a normal customer booking flow. The program was executed after the database was initialized with demo data.

MOVIE BOOKING SYSTEM - SAMPLE RUN

```
=====
MOVIE BOOKING SYSTEM
=====
1. Register
2. Login
3. Exit
Choose: 2
Email: user@example.com
Password: user123
Welcome, Demo User!

--- USER MENU ---
1. Browse movies
2. Book tickets
3. Booking history
4. Cancel booking
5. Logout
Choose: 2

Movies:
1. Campus Nights (Comedy, 7.8/10)
2. Code Runner (Thriller, 8.5/10)
3. The Last Ticket (Drama, 8.2/10)
Movie ID: 1

Shows:
3. 2026-10-01 17:00 | Screen 2 | Rs. 180.00
Show ID: 3
Available seats:
A1 A2 A3 A4 A5 A6 A7 A8 B1 B2 B3 B4 ...
Enter seats separated by comma: A1,A2

Booking confirmed! Booking ID: 1
Seats: A1, A2
Total: Rs. 360.00
```

Figure 7. Sample Customer Booking Run

The output shows login, movie selection, show selection, seat selection and booking confirmation with the final total amount.

The automated test run is shown below.

```
UNIT TEST RESULTS

Ran 5 tests

[PASS] registration_and_login
[PASS] booking_reduces_available_seats
[PASS] double_booking_is_rejected
[PASS] cancel_booking
[PASS] duplicate_email_is_rejected

Result: OK (5/5 tests passed)
```

Figure 8. Automated Test Result

11. Testing Approach

The project uses Python unittest with an isolated temporary SQLite database for each test. This prevents tests from changing the real application database. Five important functional and validation scenarios were tested.

Test Case	Expected Result	Status
Registration and login	Correct credentials return a user object.	PASS
Booking reduces available seats	Booking two seats reduces availability by two.	PASS
Double booking	A seat already booked for the same show is rejected.	PASS
Cancel booking	A confirmed booking changes to CANCELLED.	PASS
Duplicate email	A second registration with the same email is rejected.	PASS

Test command: `python -m unittest discover -s tests -v`

12. Challenges Faced

- Designing a database relationship that supports multiple seats per booking required a separate `booking_seats` table.

- Preventing double-booking required checking the current confirmed bookings before inserting the new booking.
- Keeping the code simple while still separating responsibilities required splitting the application into small service modules.
- Validation had to cover both normal mistakes, such as invalid numbers, and booking-specific mistakes, such as duplicate seats.

13. Learnings and Key Takeaways

- Learned how a real problem can be divided into smaller programming modules.
- Practiced SQL table design, foreign keys, indexes and transactions.
- Understood why validation belongs close to the data-entry boundary and why business rules belong in service modules.
- Practiced writing automated tests instead of checking every feature only by hand.
- Learned how a project repository can be documented for another person to install and run.

14. Future Enhancements

- Add a graphical or web interface for a more user-friendly experience.
- Add online payment integration and downloadable tickets.
- Add movie posters, trailers, reviews and ratings from users.
- Allow administrators to manage seat layouts and theatres from the application.
- Deploy the backend online and support multiple cinema branches.
- Add role-based permission levels and stronger account recovery features.

15. References

- VITyarthi. Build Your Own Project - General Project Instructions & Submission Guidelines. Provided course document.
- Python Software Foundation. Python 3 Documentation. <https://docs.python.org/3/>
- Python Software Foundation. unittest - Unit testing framework. <https://docs.python.org/3/library/unittest.html>
- Python Software Foundation. sqlite3 - DB-API 2.0 interface for SQLite databases. <https://docs.python.org/3/library/sqlite3.html>
- SQLite. SQLite Documentation. <https://www.sqlite.org/docs.html>

Project scope note: This submission is intentionally a local academic prototype. It demonstrates the required concepts without pretending to be a production cinema platform.